SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



SkinSage

A Multi-Agent Skincare Planner with Source-Backed, Safety-Checked Recommendations

CA-3 Project | Foundations of Agentic AI (FAAI)

Presented by:

Kavya Singh (24070126087)

Kashish Chelwani (24070126082)

Introduction

What is SkinSage?

- SkinSage is an AI-powered skincare planning system that generates personalized morning (AM) and evening (PM) skincare routines.
- It considers the user's skin type, primary concern, budget, existing products, and ingredient allergies.
- It uses multiple specialized AI agents to research skincare guidance, select suitable products, and validate ingredient compatibility.
- Every recommendation is linked to a relevant knowledge-base source and checked by a dedicated Safety Agent.

Github link

Project link

The Problem: Unsafe and Uninformed Skincare Choices

- Many people select skincare products based on advertisements, social media trends, and influencer recommendations rather than their individual needs.
- Ingredient incompatibility: Combining certain active ingredients may increase irritation or sensitivity.
- Lack of personalization: Products may not suit an individual's skin type or specific concern.
- Budget constraints: Users may spend unnecessarily on multiple overlapping products.
- Information overload: Reliable skincare guidance can be difficult to identify and apply.

Problem statement: To develop a budget-aware, personalized skincare planning system that uses multiple collaborating AI agents to retrieve relevant skincare guidance, recommend suitable products, detect potential ingredient conflicts, and validate recommendations before generating a routine.

What Does SkinSage Aim to Achieve?

1. Personalization

Generate custom AM/PM skincare routines based on individual skin type, primary concerns, user preferences, and existing products.

USER PROFILE AWARE

2. Source-Backed

Retrieve verified guidelines from a structured local knowledge base and explicitly attach Source IDs for all recommendations.

TRACEABLE EVIDENCE

3. Safety Validation

Identify ingredient conflicts, allergy matches, and unverified product claims before granting final routine approval.

AUTOMATED AUDITING

4. Budget Optimization

Select highly effective product combinations while strictly enforcing total cost constraints specified by the user.

COST CONSTRAINED

Existing Challenges vs. Proposed Solution

Existing approach	SkinSage solution
Recommendations based on trends or generic advice	Personalized routine generation
Manual checking of ingredients	Automated ingredient-conflict validation
Information scattered across sources	Local RAG-based knowledge retrieval
Products selected without budget optimization	Budget-constrained product selection
One-shot recommendation with no verification	Multi-agent review and feedback loop
Explanations without traceable references	Recommendations linked to source IDs

Innovation: SkinSage combines retrieval, product selection, safety criticism, and iterative correction in one agentic workflow.



1. Orchestrator Agent

Plans the tasks, manages shared state, executes agents in sequence, records activity, and controls retry limits.



2. Research Agent

Uses TF-IDF retrieval to find relevant skincare knowledge snippets and returns source IDs with relevance scores.



3. Product Agent

Filters the product dataset by price, skin type, concern, and avoided ingredients to build a suitable product set.



4. Safety Agent (Critic)

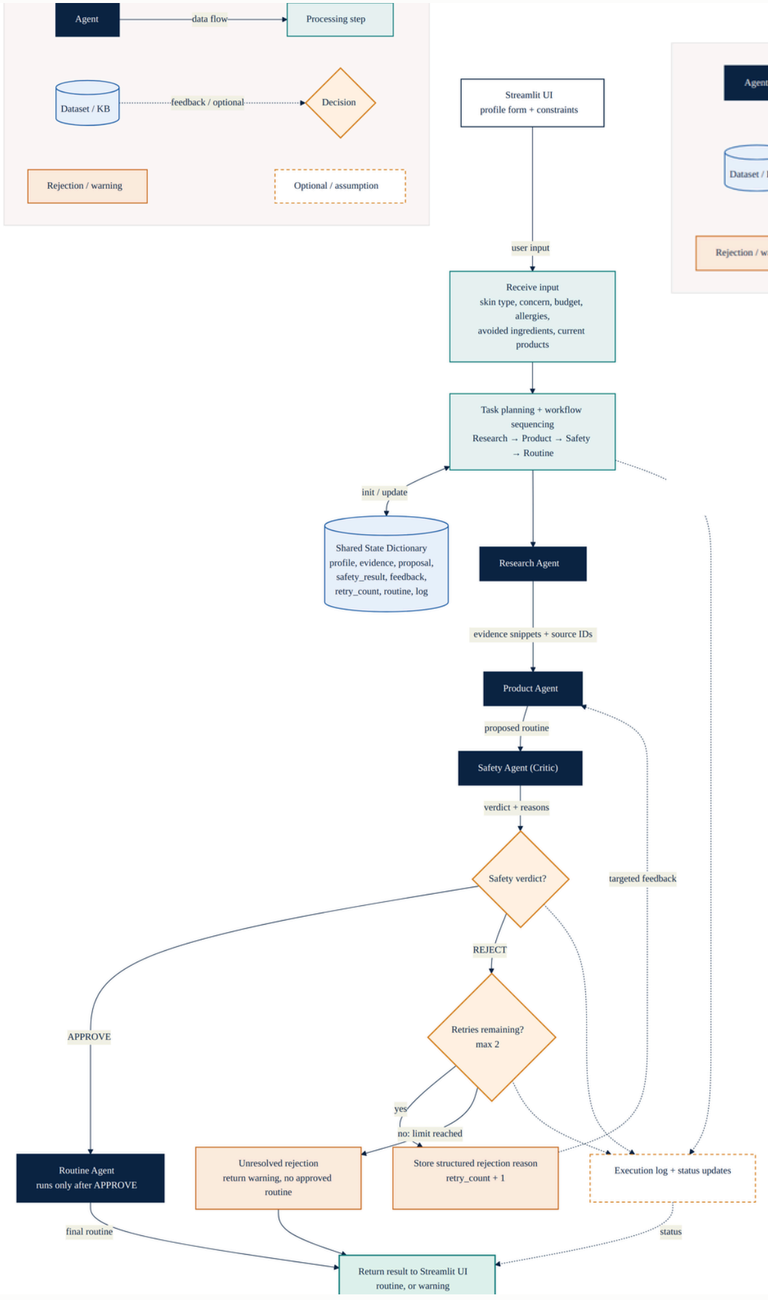
Checks ingredient conflicts, allergy matches, source coverage, and budget compliance. Rejects unsafe or invalid plans.

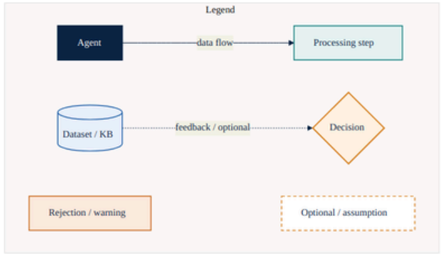


5. Routine Agent

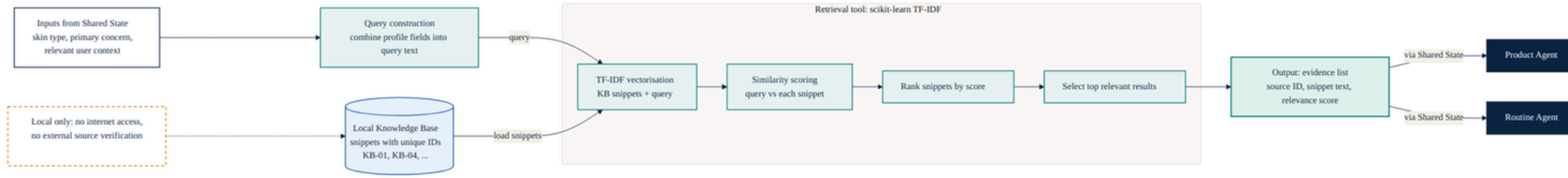
Transforms the approved product set into an organized AM/PM routine with short, source-linked explanations and a disclaimer.

1.Orchestrator

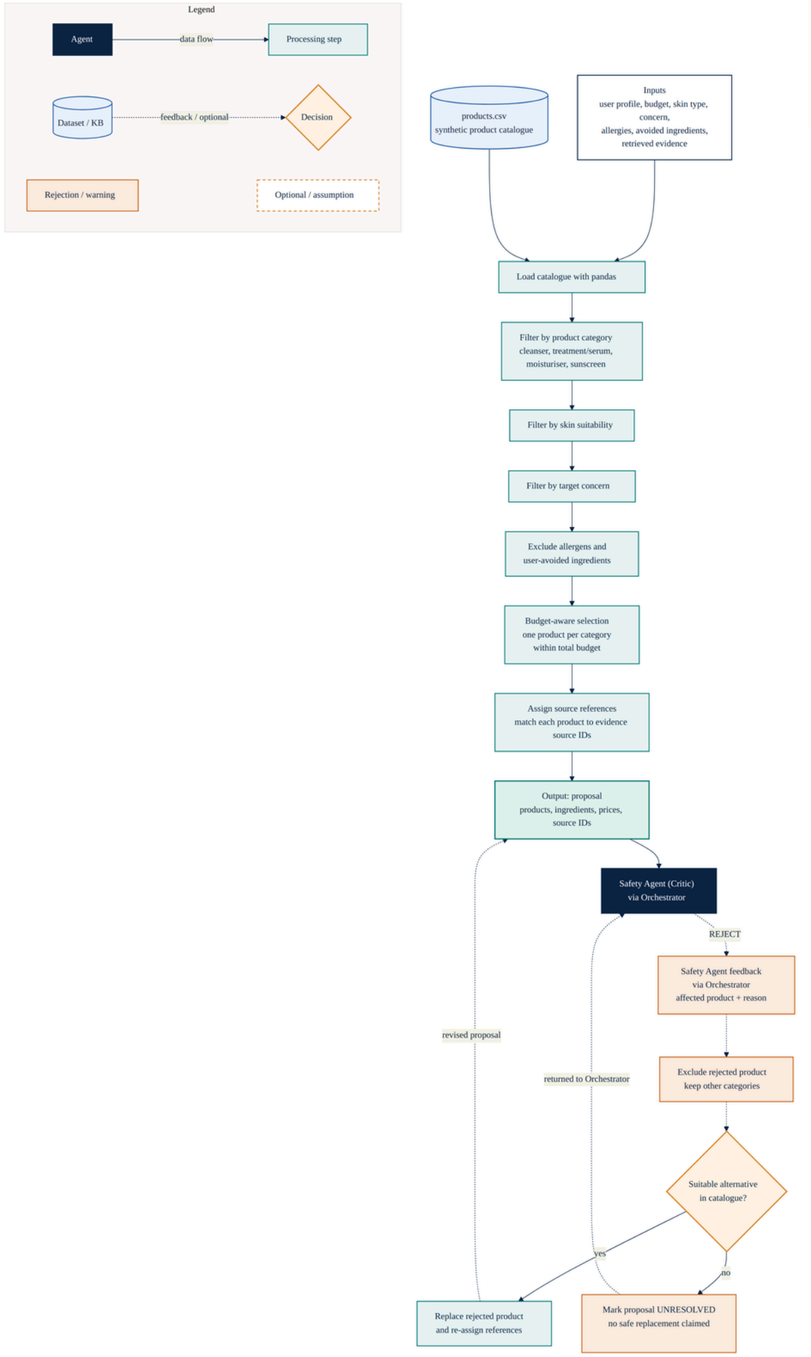




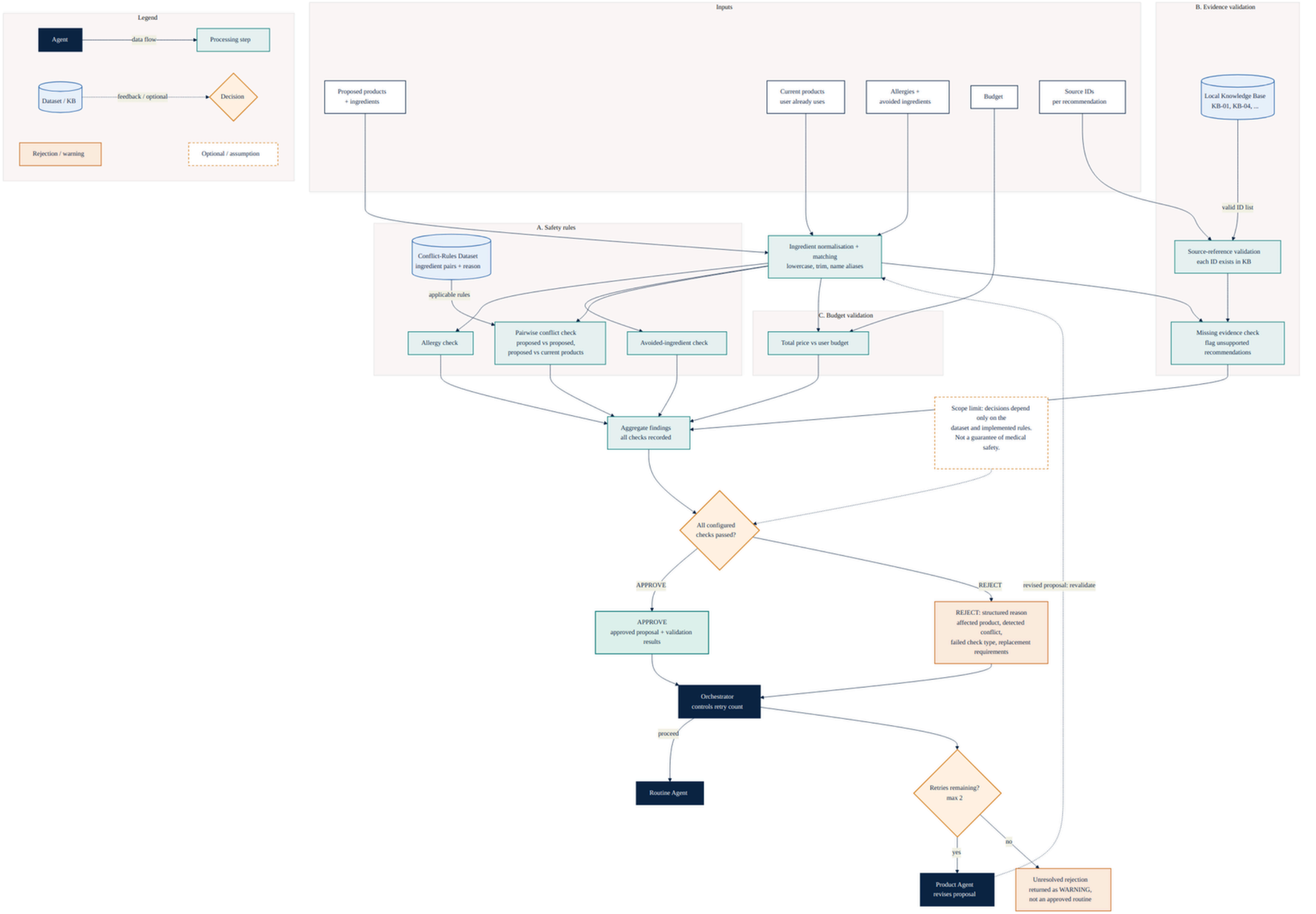
2.Research Agent



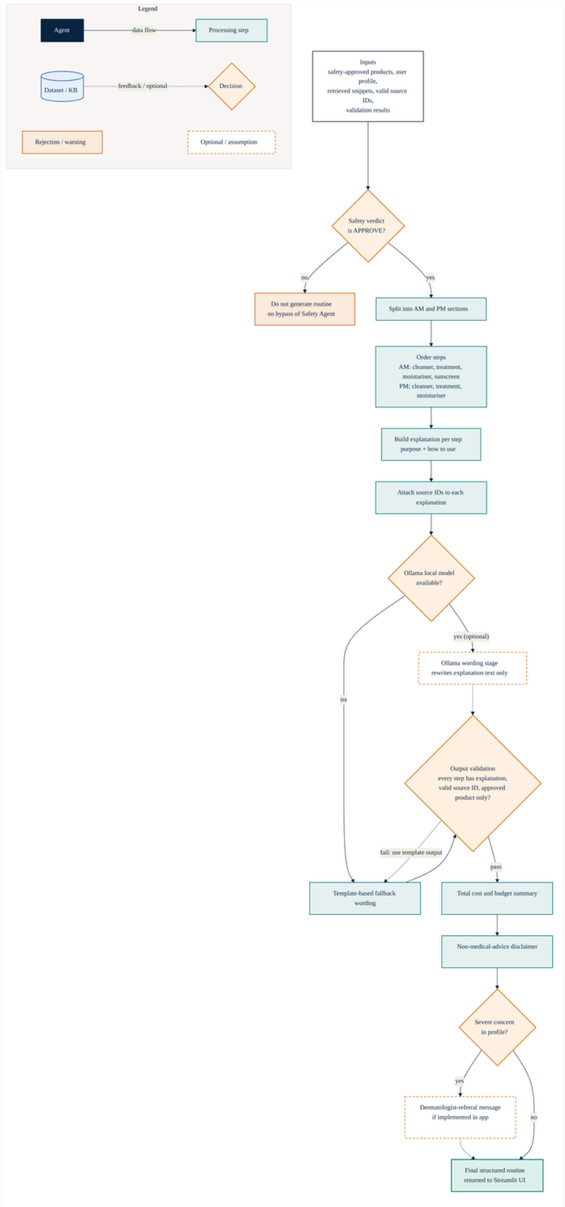
3.Product Agent



4.Safety Agent



5.Routine Agent



How the architecture works

Input layer: The user enters their skin type, concern, budget, current products and ingredients to avoid in Streamlit.

Orchestration layer: The Orchestrator creates the execution sequence and maintains the shared state.

Research layer: The Research Agent searches the local knowledge base and returns relevant evidence with source IDs.

Selection layer: The Product Agent uses that evidence and the synthetic catalogue to propose a suitable product combination.

Validation layer: The Safety Agent checks the proposal against configured safety and validation rules.

Feedback loop: A rejected proposal returns to the Product Agent with the rejection reason. The Orchestrator controls the retry limit.

Output layer: Once the proposal passes validation, the Routine Agent creates the cited AM/PM routine.

Evaluation layer: Test cases measure safety detection, retrieval quality, citation coverage and budget compliance.

ARCHITECTURAL FLOW



TECHNOLOGY STACK

- **Programming Language** – Python: Implements agents, orchestration, safety validation, and skincare routine generation.
- **Frontend** – Streamlit: Provides an interactive interface for user inputs, agent activity, and personalized recommendations.
- **Data Processing** – Pandas: Filters skincare products based on skin type, concerns, budget, and preferences.
- **Retrieval-Augmented Generation (RAG)** – Scikit-learn (TF-IDF): Retrieves relevant skincare knowledge from a local knowledge base.
- **Knowledge Base** – TXT/JSON Files: Stores skincare guidelines, ingredient information, and safety warnings with source IDs.
- **Product Database** – CSV: Contains approximately 30 synthetic skincare products with prices, ingredients, and suitability details.
- **Multi-Agent Orchestration** – Python: Coordinates agent execution, decision-making, activity logging, and feedback.
- **Safety Validation** – Rule-Based Engine: Detects ingredient conflicts, allergies, and budget violations.
- **Routine Generation** – Python Templates: Generates structured AM/PM routines with source-linked explanations and fallback support.
- **Data Storage** – CSV, JSON & Dictionaries: Stores product data, skincare knowledge, and intermediate agent outputs.
- **Evaluation & Testing** – Python: Evaluates retrieval quality, safety, citation coverage, and budget compliance.

Working of SkinSage: 5-Stage Sequence

01

Input & Planning

User submits skin type, concern, budget, existing products, and allergies. Orchestrator initializes task plan.

STEP 1: INITIALIZATION

02

Knowledge Retrieval

Research Agent searches local knowledge base via TF-IDF and retrieves relevant snippets with Source IDs.

STEP 2: EVIDENCE RAG

03

Product Selection

Product Agent selects items matching profile and budget while filtering out specified excluded ingredients.

STEP 3: CATALOG FILTER

04

Safety Review

Safety Agent checks conflicts. Detected issues feed rejection reasons back to Product Agent for revision.

STEP 4: FEEDBACK LOOP

05

Final Routine

Routine Agent formats validated AM/PM routine, source links, total cost, and required disclaimers.

STEP 5: OUTPUT GENERATION

Limitations, Future Scope & Conclusion

⚠ Current Limitations

Catalog Scale: Relies on synthetic product database (~30 items) and localized knowledge base.

Rule Ruleset: Rule-based engine cannot identify unlisted biochemical interactions.

Lexical Search: TF-IDF operates on text similarity without verifying medical evidence quality.

Non-Diagnostic: No computer vision skin analysis or clinical medical diagnosis.

Future Scope

Verified Guidelines: Connect to verified clinical dermatology guidelines and live databases.

Vision Analysis: Integrate computer vision model for automated skin condition analysis.

Tracking: Implement user longitudinal history, skin response tracking, and feedback.

Dermatologist Review: Provide human-in-the-loop dermatologist review portal.

PROJECT CONCLUSION

SkinSage successfully demonstrates a practical, safety-checked multi-agent framework that bridges information retrieval, constraint-aware optimization, and automated validation for personalized non-diagnostic skincare routines.